

Planeación de acciones para robots de servicio
mediante modelos de lenguaje y sistemas basados en
reglas

Planeación de acciones para robots de servicio a
partir de instrucciones en lenguaje natural mediante
Qwen, CLIPS y modelos GPT

Luis Sergio Cano Olguin

Tutor: Dr. Jesús Savage Carmona

Agradecimientos

Resumen

Índice general

1. Introducción	1
1.1. Contexto y Motivación	2
1.2. Planteamiento del problema	3
1.3. Hipótesis	4
1.4. Objetivos	5
1.5. Estructura del documento	6
2. Antecedentes y Estado del Arte	8
2.1. Sistemas Basados en Reglas	8
2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones	9
2.2.1. Características Fundamentales de CLIPS	10
2.3. Modelos de lenguaje para interpretación de instrucciones robóticas	11
2.3.1. Familia de modelos Qwen	12
2.3.2. Modelos GPT	13
2.3.3. Interpretación semántica y generación de planes robóticos	14
2.3.4. Limitaciones de los modelos de lenguaje en robótica	15
2.3.5. Salidas estructuradas y validación	15
2.3.6. Adaptación eficiente de modelos: LoRA y QLoRA	16
2.4. Arquitecturas neuro-simbólicas	16
2.5. Síntesis y conexión con la propuesta	19
3. Formalización del problema y arquitectura conceptual	21
3.1. Contexto arquitectónico y alcance del sistema	22
3.1.1. Integración dentro de la arquitectura ViRBot	22
3.1.2. Alcance del problema abordado	23
3.2. GPSR como dominio de referencia	24

3.2.1.	La prueba General Purpose Service Robot	24
3.2.2.	Dominio de referencia y alcance lingüístico	24
3.3.	Formalización de órdenes, objetivos y planes	25
3.3.1.	Orden en lenguaje natural	25
3.3.2.	Entidades y conocimiento del dominio	26
3.3.3.	Representación canónica de objetivos	27
3.3.4.	Objetivos semánticos y acciones ejecutables	28
3.3.5.	Dependencias y contexto entre objetivos	30
3.4.	Arquitecturas conceptuales evaluadas	31
3.4.1.	Arquitectura Qwen-CLIPS	31
3.4.2.	Arquitectura de planificación directa con GPT	32
3.5.	Criterios conceptuales de validez	33
3.5.1.	Validez de la interpretación semántica	34
3.5.2.	Validez y consistencia del plan	34
3.5.3.	Planificación y ejecución física	36
4.	Metodología: Diseño del Sistema Híbrido CLIPS-ChatGPT/Qwen	38
4.1.	Diseño del Sistema Basado en Reglas con CLIPS	38
4.1.1.	Definición del Conjunto de Reglas Jerárquicas	38
4.1.2.	Priorización de Acciones Críticas y Gestión de Emergencias . . .	38
4.1.3.	Representación del Conocimiento y Hechos en la Base de Datos de CLIPS	38
4.2.	Integración de ChatGPT/Qwen como Sistema Complementario	39
4.2.1.	Funciones Asignadas al Módulo de Lenguaje Natural	39
4.2.2.	Protocolo de Comunicación entre CLIPS y los Modelos de Lenguaje	39
4.2.3.	Mecanismos de Seguridad y Validación para las Respuestas del LLM	39
4.3.	Implementación de Planificación Dual	39
4.3.1.	Diseño del Módulo de Planificación con ChatGPT	39
4.3.2.	Protocolos de Prompting para Generación de Planes Robóticos .	39
4.3.3.	Mecanismo de Selección entre Planes CLIPS vs. ChatGPT . . .	39
4.3.4.	Validación y Simulación de Planes Antes de Ejecución	39
5.	Implementación	41
5.1.	Entornos de Desarrollo: Simulación y Plataforma Física	41

5.2.	Implementación del Motor de Reglas en CLIPS	41
5.3.	Desarrollo del Módulo de Integración	41
5.4.	Configuración de la Interfaz con API de OpenAI para ChatGPT o Modelo Qwen Local	41
5.5.	Casos de Prueba para Validar la Interacción entre los Módulos	41
6.	Escenarios de Validación y Experimentación	42
6.1.	Diseño de Experimentos en Entornos Controlados	42
6.1.1.	Escenario 1: Ejecución de Tareas Predefinidas (solo CLIPS)	42
6.1.2.	Escenario 2: Gestión de Órdenes Imprecisas o Novedosas (CLIPS + ChatGPT/Qwen)	42
6.1.3.	Escenario 3: Respuesta a Eventos Inesperados o Fallos	42
6.1.4.	Escenario 4: Planificación Dual para Tareas Complejas (CLIPS vs. ChatGPT)	42
6.2.	Métricas de Evaluación	43
6.2.1.	Métricas Cuantitativas: Tiempo de Ejecución, Tasa de Éxito, Uso de Recursos Computacionales	43
6.2.2.	Métricas Cualitativas: Robustez, Interpretabilidad y Fluidez en la Interacción Humano-Robot	43
7.	Análisis de Resultados	44
7.1.	Comparativa del Rendimiento del Sistema Solo-CLIPS vs. el Sistema Híbrido	44
7.2.	Evaluación de la Efectividad de ChatGPT/Qwen en las Diferentes Funciones Asignadas	44
7.3.	Discusión de Limitaciones y Errores	44
7.4.	Análisis de la Escalabilidad y el Coste Computacional de la Integración	44
8.	Discusión	45
8.1.	Interpretación de los Resultados en el Contexto de los Objetivos	45
8.2.	Ventajas y Desventajas del Enfoque Híbrido Propuesto	45
8.3.	Implicaciones para la Seguridad y Certificación en Entornos Regulados	45
8.4.	Límites Éticos y Prácticos del Uso de ChatGPT/Qwen en Robótica	45
9.	Contribuciones y Relevancia	46
9.1.	Contribución Técnica: Marco Híbrido Escalable y Documentado	46

9.2. Relevancia Práctica: Aplicaciones en Salud, Industria y Logística	46
9.3. Relevancia Académica: Benchmark para Sistemas Rule-Based vs. Híbridos	46
10. Conclusiones y Trabajo Futuro	47
10.1. Conclusiones Principales	47
10.2. Propuestas de Trabajo Futuro	47

Índice de figuras

3.1. Arquitectura general ViRBot del robot Justina.	22
---	----

Capítulo 1

Introducción

Los robots de servicio representan un campo de proyección para la inteligencia artificial y la automatización, con aplicaciones que van desde la asistencia a personas mayores hasta la gestión autónoma de entornos residenciales (1), donde deben interpretar instrucciones expresadas por personas y convertirlas en acciones que puedan ejecutarse mediante módulos de navegación, percepción, manipulación e interacción.(2).

La transformación de una instrucción en lenguaje natural a un plan robótico presenta dos problemas relacionados. El primero es semántico: identificar correctamente los objetivos, entidades, destinos y relaciones expresados por el usuario. El segundo es simbólico: convertir dicha interpretación en una secuencia de acciones pertenecientes al repertorio real del robot y con parámetros compatibles con el ejecutor.

Los sistemas basados en reglas, han demostrado ser robustos y predecibles en escenarios estructurados, como líneas de producción o laboratorios controlados (3). En particular, CLIPS utiliza hechos y reglas de producción para determinar qué transformaciones deben aplicarse a partir de un estado simbólico. Sin embargo, su rigidez los hace poco adaptables ante variaciones no previstas en el entorno o ante comandos expresados en lenguaje natural con alto grado de ambigüedad o complejidad (4).

Recientemente, los modelos de lenguaje grande (LLMs), como ChatGPT, han emergido como herramientas capaces de interpretar y generar lenguaje natural, e incluso de actuar como agentes de planificación autónomos (5). Su flexibilidad contextual permite traducir instrucciones verbales en secuencias de acciones, complementando así la solidez de los sistemas basados en reglas. No obstante, su naturaleza probabilística puede producir formatos inválidos, entidades inexistentes o acciones que no pertenecen a las capacidades del robot. Por lo tanto la falta de garantías formales de seguridad limitan

su uso directo en aplicaciones robóticas donde la integridad física y la predictibilidad son prioritarias (6).

Esta tesis estudia dos aproximaciones para resolver el problema. La primera es una arquitectura neuro-simbólica en la que un modelo Qwen2.5-7B, ajustado mediante QLoRA, transforma una orden en objetivos canónicos representados en JSON. Posteriormente, un módulo determinista normaliza y convierte esos objetivos en hechos, que son procesados por reglas de producción en CLIPS para generar el plan compatible con el ejecutor del robot. La segunda aproximación emplea un modelo GPT como planificador generativo directo. En este caso, el modelo recibe la orden, el contexto y el catálogo de acciones, produciendo una respuesta estructurada mediante JSON Schema que es revisada por un validador determinista antes de considerarse ejecutable.

Las dos aproximaciones no se combinan mediante un selector de planes durante la ejecución. Se implementan como arquitecturas independientes y se comparan experimentalmente bajo el mismo conjunto de órdenes, contexto y criterios de evaluación. Esta comparación permite analizar el compromiso entre interpretación semántica, consistencia, trazabilidad, flexibilidad, latencia, recursos computacionales y coste de uso.

1.1. Contexto y Motivación

El área de la robotica orientada al servicio doméstico ha evolucionado significativamente en la última década, impulsada por avances en percepción, planificación y interacción humano-robot (1).

El robot de servicio Justina, desarrollado en el Laboratorio de Bio-Robótica de la UNAM, integra módulos de percepción, navegación, manipulación e interacción mediante ROS 2. Para ejecutar una orden, el robot necesita una representación intermedia que conecte el lenguaje del usuario con las acciones ofrecidas por dichos módulos.

Estos sistemas se han diseñado para operar en entornos no estructurados como hogares, hospitales o centros de cuidado, donde deben realizar tareas que van desde la entrega de objetos hasta la asistencia en actividades de la vida diaria. En este contexto, la capacidad de planificar acciones de manera autónoma, segura y adaptativa se convierte en un requisito fundamental.

La planificación de acciones en robótica ha sido tradicionalmente abordada mediante sistemas basados en reglas (*Rule-Based Systems, RBS*), que ofrecen un marco predecible y verificable para la toma de decisiones (2). Sin embargo, una solución exclusivamente

simbólica puede generar planes consistentes cuando recibe objetivos correctamente estructurados, pero no resuelve directamente la interpretación del lenguaje natural. En contraste, un modelo de lenguaje puede interpretar distintas formulaciones de una orden, aunque su salida no es necesariamente compatible con las restricciones del robot. La motivación de este trabajo es evaluar si la separación entre interpretación neuronal y expansión simbólica permite obtener planes ejecutables y trazables, y contrastar sus resultados con los de un planificador basado exclusivamente en un modelo GPT.

El interés de la comparación no consiste en demostrar que un paradigma es universalmente superior. Se busca identificar que arquitectura presenta ventajas dependiendo del comando, así como que errores introduce cada etapa y cuáles son los costes asociados con su despliegue local o mediante una API externa.

1.2. Planteamiento del problema

Una orden puede contener variaciones lingüísticas, referencias espaciales, objetos, personas, destinos y varias subtarefas relacionadas. Para que el robot pueda ejecutarla, la orden debe convertirse en una secuencia válida de acciones primitivas, para esto el sistema basados en reglas CLIPS, fundamentado en la lógica simbólica y el ciclo *reconocer-actuar* (3), usa un conjunto de hechos y reglas bien definidos para producir una respuesta determinista y explicable, lo que lo hace idóneo para aplicaciones donde la seguridad es crítica, como entornos industriales o médicos.

La conversión por parte de CLIPS enfrenta los siguientes problemas:

1. **Rigidez interpretativa** No se pueden procesar comandos en lenguaje natural complejo o ambiguo.
2. **Falta de adaptabilidad:** Las reglas deben ser definidas a priori; cualquier situación no prevista en la base de conocimiento puede llevar al fracaso o a un comportamiento no deseado.
3. **Dificultad para gestionar la incertidumbre:** No manejan bien la información incompleta o cambiante del entorno en tiempo real.
4. **Escalabilidad limitada:** Mantener y extender un conjunto grande de reglas para cubrir todos los escenarios posibles resulta complejo y laborioso.

Por otro lado, los resultados deben satisfacer al menos las siguientes condiciones (6):

- Las entidades finales deben pertenecer al catálogo o ser tratadas explícitamente como desconocidas.

- Los objetivos deben conservar el significado y el orden causal de la instrucción.
- Las acciones generadas deben pertenecer al repertorio del ejecutor.
- Cada acción debe contener los parámetros requeridos.
- La secuencia debe respetar precondiciones simbólicas básicas, como la ubicación del robot o la posesión de un objeto.

Los modelos de lenguaje pueden ampliar la cobertura lingüística, aunque también pueden generar resultados inexistentes o inconsistentes. El problema de investigación se formula, por tanto, de la siguiente manera:

¿Qué diferencias de exactitud, consistencia, robustez y coste existen entre una arquitectura neuro-simbólica Qwen-CLIPS y un planificador generativo basado en GPT al convertir órdenes GPSR en planes para un robot de servicio?

1.3. Hipótesis

Para la interpretación y planeación de órdenes, una arquitectura neuro-simbólica compuesta por Qwen y CLIPS producirá una mayor proporción de planes estructuralmente válidos, trazables y consistentes que un planificador generativo basado directamente en GPT. Por otra parte, se espera que el planificador GPT directo presente mayor flexibilidad ante formulaciones lingüísticas novedosas, pero con mayor variabilidad, latencia y coste de inferencia mediante API.

Se espera que los sistemas:

- Mejore la tasa de éxito en la ejecución de tareas en entornos domésticos.
- Reduzca el tiempo de respuesta ante comandos complejos.
- Mantenga un comportamiento seguro y predecible en situaciones críticas.
- Permita una interacción más natural e intuitiva con usuarios no expertos.

La hipótesis se evaluará de forma separada para la interpretación de objetivos y para la generación del plan completo. De esta manera será posible determinar si un error proviene del modelo de lenguaje, de la conversión simbólica o de la cobertura de las reglas CLIPS.

1.4. Objetivos

Diseñar, implementar y evaluar una arquitectura neuro-simbólica Qwen–CLIPS para interpretar órdenes y generar planes compatibles con un robot de servicio, comparándola con un planificador generativo basado en GPT con salida estructurada.

■ Objetivos específicos

- Definir una representación canónica de objetivos y su correspondencia con las acciones disponibles en el ejecutor.
- Ajustar Qwen2.5-7B mediante QLoRA para convertir órdenes en objetivos canónicos representados en JSON.
- Implementar la normalización, validación y conversión de los objetivos generados por Qwen en hechos.
- Implementar reglas de producción en CLIPS que expandan objetivos de alto nivel en secuencias de acciones compatibles con el ejecutor.
- Diseñar un planificador GPT directo mediante un prompt, un JSON Schema y un validador determinista.
- Construir un conjunto experimental que incluya órdenes conocidas, paráfrasis, composiciones de objetivos, entradas ambiguas y casos fuera del dominio.
- Comparar las arquitecturas en términos de exactitud semántica, validez estructural, robustez, latencia, consumo de recursos, variabilidad y coste.
- Validar en simulación, robot físico y durante la RoboCup 2026 el resultado los planes generados.

■ Contribuciones esperadas

- Una representación intermedia de objetivos que conecta la interpretación lingüística con la planeación simbólica.
- Un modelo Qwen2.5-7B ajustado mediante QLoRA para traducir órdenes a objetivos canónicos.
- Una integración ROS 2–CLIPS que convierte dichos objetivos en planes compatibles con el ejecutor.
- Un planificador GPT directo restringido mediante prompt, JSON Schema y validación determinista.
- Un protocolo experimental reproducible para comparar planeación neuro-simbólica y planeación generativa.
- Un análisis de errores por etapa que diferencia fallos semánticos, estructurales, simbólicos y de ejecución.

1.5. Estructura del documento

Este documento está organizado en los siguientes capítulos que describen el sistema propuesto para el desarrollo de esta tesis, abarcando el planteamiento, desarrollo, implementación y validación del sistema híbrido de planificación. La estructura es la siguiente:

1. **Capítulo 1: Introducción.** Presenta el contexto, motivación, problemática, hipótesis y objetivos de la investigación, así como la justificación del enfoque híbrido en robótica de servicio doméstico.
2. **Capítulo 2: Antecedentes y estado del arte.** Revisa la evolución de los sistemas basados en reglas en robótica, las características de CLIPS, los enfoques modernos de aprendizaje automático y los sistemas híbridos existentes.
3. **Capítulo 3: Marco teórico y conceptual.** Describe los fundamentos de la planificación jerárquica, los sistemas de producción, los modelos de lenguaje grande (LLMs) y la arquitectura de integración propuesta.
4. **Capítulo 4: Metodología.** Detalla el diseño del sistema híbrido, incluyendo la definición de reglas en CLIPS, la integración con ChatGPT/Qwen, y los mecanismos de planificación dual y validación.
5. **Capítulo 5: Implementación.** Explica los entornos de desarrollo, la configuración técnica, la interfaz con APIs y las pruebas utilizados para validar la integración de los módulos.
6. **Capítulo 6: Escenarios de validación y experimentación.** Presenta el diseño experimental, los escenarios de prueba y las métricas cuantitativas y cualitativas utilizadas para evaluar el sistema.
7. **Capítulo 7: Análisis de resultados.** Compara el rendimiento del sistema híbrido frente al sistema basado solo en reglas, evalúa la efectividad de los LLMs y discute limitaciones y costos computacionales.
8. **Capítulo 8: Discusión.** Interpreta los resultados en relación con los objetivos, analiza ventajas y desventajas del enfoque híbrido, y explora implicaciones éticas y de seguridad.
9. **Capítulo 9: Contribuciones y relevancia.** Destaca la aportación técnica del marco híbrido, su aplicabilidad práctica en distintos sectores y su relevancia académica como benchmark.
10. **Capítulo 10: Conclusiones y trabajo futuro.** Resume los hallazgos principales y propone líneas de investigación futura, como el fine-tuning de LLMs y la

mejora de mecanismos de seguridad.

Capítulo 2

Antecedentes y Estado del Arte

Este capítulo presenta los trabajos y tecnologías relacionados con la transformación de instrucciones de lenguaje natural a planes para robots de servicio. La revisión se concentra en cuatro componentes relevantes para la tesis: sistemas basados en reglas, CLIPS, modelos de lenguaje para interpretación y planificación, y arquitecturas neuro-simbólicas. Los métodos de percepción, navegación o control se mencionan únicamente cuando contribuyen directamente a la interfaz entre la orden y el plan de tareas.

2.1. Sistemas Basados en Reglas

Los sistemas basados en reglas (*Rule-Based Systems, RBS*) constituyen uno de los paradigmas clásicos de la inteligencia artificial (3). Su utilización se consolidó con el desarrollo de los sistemas expertos durante las décadas de 1970 y 1980, en los que el conocimiento de especialistas humanos era representado explícitamente mediante hechos y reglas dentro de dominios bien delimitados (2). En robótica, la adopción de estos sistemas se popularizó en los años 80 y 90, particularmente en entornos industriales estructurados, donde la previsibilidad y seguridad eran prioritarias.

Este tipo de representación resulta útil cuando las condiciones bajo las cuales deben ejecutarse determinadas acciones pueden expresarse de manera explícita. Las reglas permiten relacionar estados, eventos u objetivos con decisiones específicas, facilitando la construcción de comportamientos reproducibles dentro de dominios previamente definidos. Entre las herramientas desarrolladas para implementar este paradigma se encuentra CLIPS (*C Language Integrated Production System*), desarrollado originalmente por la NASA. CLIPS proporciona mecanismos para representar conocimiento mediante

hechos y reglas de producción, junto con un motor de inferencia encargado de determinar qué reglas deben activarse de acuerdo con el estado de la base de conocimiento (7).

En cuanto a aplicaciones, los sistemas basados en reglas han demostrado su utilidad en diversos dominios robóticos:

1. **Robótica industrial:** Cadenas de montaje automatizadas, donde las tareas son repetitivas y el entorno está controlado. Los RBS se utilizan para secuenciar operaciones, gestionar excepciones y garantizar la seguridad de los operarios humanos (1).
2. **Robótica de servicio y doméstica:** En tareas simples como la entrega de objetos, navegación en interiores y asistencia a personas con movilidad reducida.
3. **Robótica espacial y de exploración:** Sistemas como Remote Agent de la NASA emplearon arquitecturas basadas en reglas para la planificación y ejecución autónoma de experimentos en misiones no tripuladas (8).

Su principal ventaja para esta tesis es que permiten establecer una correspondencia explícita entre un objetivo simbólico y las acciones emitidas para el robot. Sin embargo, la cobertura del sistema está limitada por las reglas y los tipos de hechos definidos previamente.

2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones

CLIPS es un lenguaje de programación basado en reglas, diseñado específicamente para la construcción de sistemas expertos y aplicaciones basadas en conocimiento (7). Concebido originalmente como una alternativa a los costosos sistemas implementados en (*LISP*, *LISt Processing*) en estaciones de trabajo especializadas, CLIPS destacó por su portabilidad, eficiencia y su capacidad para ejecutarse en hardware de bajo costo (3). Su adopción se extendió rápidamente en la comunidad de robótica e inteligencia artificial, convirtiéndose en una herramienta de referencia para la implementación de sistemas de producción (2).

2.2.1. Características Fundamentales de CLIPS

CLIPS es un entorno para el desarrollo de sistemas basados en conocimiento que ofrece hechos, plantillas, reglas de producción, funciones y un motor de inferencia que permite razonar sobre una base de conocimiento mediante reglas de producción (3).

En el contexto de esta tesis, las propiedades relevantes de CLIPS son:

1. **Encadenamiento hacia adelante (*forward chaining*):** CLIPS utiliza predominantemente encadenamiento hacia adelante. A partir de los hechos presentes en la base de conocimiento, el motor identifica las reglas cuyas condiciones se satisfacen y las incorpora a la agenda de ejecución. La ejecución de estas reglas puede modificar posteriormente la base de hechos y habilitar nuevas reglas (7).
2. **Representación explícita del conocimiento:** Además de reglas, CLIPS permite representar conocimiento mediante hechos ordenados y plantillas (*deftemplate*), así como mediante funciones definidas por el usuario en C (7). Esto facilita la integración con sistemas de percepción y control de bajo nivel (9).
3. **Trazabilidad de las reglas activadas:** Debido a que las decisiones dependen de hechos y reglas explícitas, es posible inspeccionar las reglas que son activadas durante una ejecución. Cuando se mantienen constantes los hechos iniciales, las reglas y la estrategia de resolución de conflictos, el comportamiento del motor puede reproducirse de manera determinista.
4. **Portabilidad:** CLIPS fue implementado en C con el propósito de facilitar su utilización en diferentes plataformas. Esta característica permite incorporar el motor de inferencia dentro de sistemas de software más amplios sin depender de una plataforma especializada (3,7).
5. **Integración con otros lenguajes:** CLIPS proporciona interfaces que permiten incorporarlo dentro de aplicaciones desarrolladas en otros lenguajes. En el caso de Python, esta comunicación puede realizarse mediante bindings externos que exponen las funciones del motor de CLIPS. Esto permite utilizar el sistema basado en reglas como un componente dentro de una arquitectura robótica más amplia.

Estas propiedades contribuyen a restringir y validar el comportamiento del sistema, pero no implican, por sí mismas, una garantía de seguridad ni constituyen un mecanismo de verificación formal.

CLIPS tampoco interpreta directamente lenguaje natural ni aprende reglas a partir de los ejemplos empleados en esta investigación. Para recibir una orden requiere una etapa externa que produzca objetivos estructurados. Además, la integración con ROS

2 no es una capacidad intrínseca de CLIPS, sino una interfaz implementada específicamente para comunicar el motor de reglas con los nodos del robot.

2.3. Modelos de lenguaje para interpretación de instrucciones robóticas

El desarrollo del aprendizaje automático, y en particular del aprendizaje profundo, ha ampliado las capacidades de los sistemas robóticos para procesar información compleja y entradas con un alto grado de variabilidad (10). A diferencia de los sistemas basados en reglas, en los que parte del conocimiento del dominio debe representarse explícitamente, los enfoques de aprendizaje automático permiten obtener representaciones y patrones a partir de datos (2).

En el procesamiento de información secuencial, arquitecturas como las redes neuronales recurrentes (RNN), LSTM y GRU permitieron modelar dependencias entre los elementos de una secuencia y fueron utilizadas en diferentes tareas relacionadas con el procesamiento del lenguaje natural (10).

Posteriormente, la arquitectura Transformer introdujo un enfoque basado principalmente en mecanismos de atención, permitiendo modelar relaciones entre los elementos de una secuencia sin depender exclusivamente de recurrencia (5). Esta arquitectura constituye la base de gran parte de los modelos de lenguaje de gran escala desarrollados posteriormente.

Los *Large Language Models* (LLMs) basados en Transformers pueden procesar y generar lenguaje natural utilizando el contexto de una instrucción para producir diferentes tipos de representaciones. En robótica, estos modelos se han utilizado para interpretar comandos complejos, traducir lenguaje natural a representaciones simbólicas e incluso generar planes a partir de descripciones de tareas de alto nivel (6).

Estas capacidades resultan relevantes para esta investigación debido a que permiten utilizar un modelo de lenguaje como interfaz entre las expresiones proporcionadas por el usuario y las representaciones estructuradas utilizadas posteriormente por los componentes de planificación.

2.3.1. Familia de modelos Qwen

Qwen es una familia de modelos de lenguaje desarrollada por el equipo Qwen de Alibaba Cloud. La familia incluye modelos de diferentes tamaños y variantes destinadas a tareas generales y al seguimiento de instrucciones (13). Estos modelos están basados en la arquitectura Transformer y pueden utilizarse en tareas como generación de texto, respuesta a preguntas, extracción de información y producción de contenido estructurado.

Qwen2.5 corresponde a una generación de esta familia que incluye modelos con diferentes números de parámetros. Para esta investigación se utiliza **Qwen/Qwen2.5-7B** como componente neuronal encargado de la interpretación de las instrucciones proporcionadas al robot. Los criterios que motivaron la selección de esta variante se desarrollan posteriormente.

En la arquitectura propuesta, Qwen2.5-7B funciona como intérprete semántico y no como generador del plan final. El modelo recibe una orden expresada en lenguaje natural y produce una secuencia de objetivos canónicos representada en formato JSON. Esta secuencia identifica los tipos de objetivo, las entidades, las ubicaciones y las relaciones extraídas de la instrucción. Posteriormente, un módulo determinista normaliza y valida la respuesta antes de convertirla en hechos que pueden ser procesados por CLIPS.

De esta manera, el modelo de lenguaje se utiliza específicamente para reducir la variabilidad lingüística presente en las instrucciones, mientras que la generación de las acciones necesarias para alcanzar los objetivos permanece bajo responsabilidad del componente simbólico.

Justificación de la selección de Qwen2.5-7B

La selección de Qwen2.5-7B como modelo base responde tanto a las características requeridas por la tarea como a las restricciones computacionales disponibles durante el desarrollo de esta investigación. La tarea planteada requiere interpretar instrucciones expresadas en lenguaje natural y transformarlas en una representación estructurada de objetivos canónicos, por lo que resulta conveniente utilizar un modelo con capacidad suficiente para seguir instrucciones, conservar las relaciones semánticas presentes en un orden y generar salidas con una estructura consistente.

Durante las etapas preliminares de desarrollo se realizaron pruebas con Qwen2.5-0.5B debido a sus menores requerimientos computacionales y a la posibilidad de efectuar rápidamente iteraciones de ajuste y evaluación. Estas pruebas tuvieron un carácter

exploratorio y permitieron validar inicialmente el flujo de entrenamiento y la representación utilizada para los objetivos.

Sin embargo, durante estas pruebas se observó que las respuestas del modelo presentaban una consistencia insuficiente para los requerimientos de la tarea, particularmente ante instrucciones con mayor complejidad o variabilidad lingüística. Debido a ello, se optó por utilizar una variante con mayor capacidad para los experimentos posteriores.

Estas observaciones iniciales se utilizaron como criterio de diseño para seleccionar la siguiente variante del modelo y no constituyen, por sí mismas, una comparación exhaustiva entre los diferentes tamaños de la familia Qwen2.5.

Se seleccionó finalmente Qwen2.5-7B como un compromiso entre capacidad del modelo y viabilidad computacional. Bajo la configuración de entrenamiento utilizada y los recursos computacionales disponibles durante esta investigación, la variante de siete mil millones de parámetros representó el mayor tamaño que resultó práctico para realizar de manera local los procesos de ajuste y experimentación. La plataforma utilizada estuvo compuesta por una GPU NVIDIA GeForce RTX 4070 Ti, un procesador Intel Core i7 de undécima generación y 32 GB de memoria RAM.

La utilización de modelos de mayor tamaño habría incrementado los requerimientos de memoria y procesamiento, además de reducir la flexibilidad para realizar múltiples iteraciones experimentales dentro de la infraestructura disponible. Por esta razón, se optó por mantener el modelo dentro de un tamaño que permitiera realizar tanto su adaptación como su evaluación localmente.

La elección de Qwen2.5-7B no pretende establecer que este modelo sea superior de manera general a otras familias de modelos de lenguaje. Existen diversas alternativas que podrían emplearse para una tarea de este tipo; sin embargo, la selección se realizó considerando el equilibrio entre capacidad para procesar instrucciones, disponibilidad del modelo, posibilidad de ejecutarlo y adaptarlo localmente, y los recursos computacionales disponibles. De esta manera, Qwen2.5-7B constituye una solución viable para evaluar la integración de un modelo de lenguaje especializado dentro de la arquitectura neuro-simbólica propuesta.

2.3.2. Modelos GPT

Los modelos GPT constituyen otra familia de modelos generativos basados en la arquitectura Transformer. En esta tesis, el modelo seleccionado se utiliza mediante la API de OpenAI, en lugar de desplegar sus parámetros dentro de la infraestructura local.

La API recibe la instrucción y el contexto proporcionado para la tarea y devuelve una respuesta generada por el modelo.

En el capítulo de implementación se especifican el modelo y la configuración utilizados durante los experimentos, debido a que una denominación general como “ChatGPT” no identifica de manera reproducible un modelo específico accesible mediante API (14).

En la arquitectura experimental GPT directo, el modelo recibe la orden, el catálogo de entidades, el repertorio de acciones, las restricciones del dominio y ejemplos de respuesta. A diferencia de Qwen implementado en la arquitectura neuro-simbólica, este modelo genera tanto la representación de los objetivos como la secuencia propuesta de acciones. Por ello, constituye un planificador generativo independiente y no un módulo interno de la cadena Qwen-CLIPS.

La respuesta del modelo se solicita mediante una salida estructurada definida a partir de JSON Schema. La documentación de la API denomina *Structured Outputs* al mecanismo mediante el cual la respuesta generada debe ajustarse al esquema proporcionado (14). Este mecanismo facilita la integración de la salida con los componentes posteriores del sistema, aunque el cumplimiento del esquema no garantiza por sí mismo la validez semántica del contenido generado.

Los distintos niveles de validación empleados para las salidas de los modelos se describen posteriormente en la subsección de salidas estructuradas y validación.

El uso de una API externa introduce variables que deben incluirse en la evaluación, como latencia de comunicación, disponibilidad del servicio, tokens de entrada y salida, coste monetario y posibles cambios entre versiones del modelo. En consecuencia, las pruebas deben fijar y documentar el modelo, los parámetros de generación y el prompt.

2.3.3. Interpretación semántica y generación de planes robóticos

Para esta tesis conviene distinguir dos funciones:

Interpretación semántica: Transformación de una orden en una representación intermedia de objetivos, entidades y relaciones.

Planificación generativa: Producción directa de una secuencia de acciones a partir de la orden, el contexto y las capacidades declaradas del robot.

En la arquitectura Qwen-CLIPS estas funciones se mantienen separadas: Qwen realiza la interpretación semántica de la orden y CLIPS expande los objetivos obtenidos

en una secuencia de acciones. En la arquitectura GPT directo, ambas funciones se concentran en un único modelo, que produce tanto la representación de los objetivos como el plan propuesto. Esta distinción permite comparar el efecto de mantener una etapa simbólica explícita frente a delegar conjuntamente la interpretación y la composición del plan a un modelo generativo.

La interpretación semántica puede evaluarse comparando la secuencia de objetivos producida por el modelo con una anotación de referencia. La planificación requiere criterios adicionales: pertenencia de las acciones al repertorio del robot, validez de parámetros, conservación de dependencias, cumplimiento de precondiciones y compatibilidad con el ejecutor. Evaluar ambas etapas de forma separada permite localizar el origen de los errores.

2.3.4. Limitaciones de los modelos de lenguaje en robótica

El uso de un modelo de lenguaje introduce riesgos específicos: generación de entidades o acciones inexistentes, incumplimiento del formato, variabilidad entre ejecuciones, sensibilidad al *prompt* y costes de inferencia. Asimismo, una instrucción ambigua puede dar lugar a una interpretación plausible pero diferente de la intención del usuario.

Estas limitaciones son especialmente relevantes cuando la salida se utiliza para controlar un sistema físico. Por ello, una respuesta lingüísticamente coherente no debe considerarse automáticamente un plan ejecutable. En esta investigación se separan la generación, la validación estructural y la validación semántica; los módulos de ejecución del robot conservan la responsabilidad sobre restricciones físicas y condiciones observadas durante la tarea.

2.3.5. Salidas estructuradas y validación

Una estrategia para integrar modelos de lenguaje con software convencional consiste en restringir la salida a una estructura legible por máquina. JSON permite representar objetivos, parámetros y acciones, mientras que JSON Schema permite comprobar tipos, campos obligatorios, enumeraciones y restricciones estructurales.

No obstante, una respuesta conforme al esquema puede seguir siendo semánticamente incorrecta. Por ejemplo, puede incluir una ubicación que no pertenece al mapa o aplicar una acción válida al tipo de entidad equivocado. En consecuencia, la validación debe separar al menos tres niveles:

1. Sintaxis JSON.
2. Conformidad con el esquema.
3. Validez semántica respecto a catálogos, acciones y precondiciones.

Esta separación es aplicable tanto a los objetivos producidos por Qwen como al plan generado directamente por GPT.

2.3.6. Adaptación eficiente de modelos: LoRA y QLoRA

El ajuste completo de un modelo de lenguaje requiere actualizar todos sus parámetros y mantener estados adicionales durante el entrenamiento. Los métodos de adaptación eficiente en parámetros permiten reducir este coste al entrenar únicamente un subconjunto reducido de parámetros, manteniendo congelada gran parte del modelo preentrenado (15, 16). LoRA (*Low-Rank Adaptation*) introduce matrices entrenables de bajo rango en determinadas capas del modelo, manteniendo congelados los pesos originales (16), mientras que QLoRA combina estos adaptadores con la cuantización del modelo base para reducir significativamente el uso de memoria durante el ajuste fino (17).

En esta investigación se utiliza QLoRA para especializar Qwen2.5-7B en la conversión de órdenes a objetivos canónicos. Los detalles del conjunto de datos, cuantización, hiperparámetros y plataforma pertenecen al capítulo de implementación.

2.4. Arquitecturas neuro-simbólicas

Las arquitecturas neuro-simbólicas buscan combinar las capacidades de aprendizaje y generalización de los modelos neuronales con mecanismos explícitos de representación de conocimiento y razonamiento simbólico. Mientras que los modelos neuronales son especialmente adecuados para procesar información no estructurada y aprender representaciones a partir de datos, los sistemas simbólicos permiten expresar conocimiento mediante estructuras discretas, reglas y relaciones que pueden ser utilizadas durante un proceso de inferencia (18, 19). El objetivo de este enfoque es aprovechar las fortalezas de ambos paradigmas en lugar de utilizar cada uno de manera aislada.

La integración entre ambos componentes puede realizarse de diferentes maneras. Un modelo neuronal puede, por ejemplo, transformar una entrada no estructurada en una representación simbólica que posteriormente es procesada por un sistema de razonamiento. En la dirección opuesta, el conocimiento simbólico puede utilizarse para

imponer restricciones, validar resultados o proporcionar información adicional durante el aprendizaje o la inferencia del modelo neuronal (19). Por tanto, una arquitectura neuro-simbólica no implica necesariamente que el aprendizaje neuronal y el razonamiento simbólico se realicen dentro de un único modelo.

Esta separación resulta particularmente útil cuando la entrada del sistema se encuentra expresada en lenguaje natural, pero la ejecución posterior requiere representaciones estructuradas y operaciones sujetas a reglas explícitas. En este tipo de arquitecturas, el modelo neuronal puede encargarse de interpretar la variabilidad lingüística y transformarla en una representación formal o canónica, mientras que un componente simbólico utiliza dicha representación para realizar razonamiento o planificación.

Un ejemplo de integración entre modelos de lenguaje y planificación simbólica es la arquitectura LLM+P, propuesta en *Empowering Large Language Models with Optimal Planning Proficiency* (20). En este enfoque, el modelo de lenguaje no se utiliza directamente como el mecanismo encargado de buscar la secuencia final de acciones. En su lugar, participa en la transformación de una descripción expresada en lenguaje natural hacia una representación formal que posteriormente puede ser procesada por un planificador clásico.

Para esta representación se utiliza PDDL (*Planning Domain Definition Language*), un lenguaje destinado a describir problemas de planificación automática. De manera general, PDDL permite representar los elementos que caracterizan al dominio de planificación, como las acciones disponibles, sus parámetros, precondiciones y efectos, así como los elementos específicos de un problema, entre ellos los objetos involucrados, el estado inicial y el objetivo que debe alcanzarse.

Una vez obtenida la representación formal del problema, la búsqueda de una secuencia de acciones que satisfaga el objetivo se delega a un planificador clásico. De esta manera, el modelo de lenguaje participa principalmente en la interpretación y formalización de la descripción proporcionada, mientras que el razonamiento necesario para obtener el plan se realiza mediante un mecanismo simbólico.

De manera simplificada, este flujo puede representarse como:

Lenguaje natural $\rightarrow$ Modelo de lenguaje $\rightarrow$ PDDL $\rightarrow$ Planificador clásico $\rightarrow$ Plan.

Esta separación permite asignar a cada componente una función distinta: el modelo de lenguaje se encarga principalmente de interpretar expresiones lingüísticas y convertirlas

en una representación estructurada, mientras que el planificador trabaja sobre dicha representación para determinar una secuencia de acciones que satisfaga las restricciones del problema (20). Este enfoque constituye un ejemplo de cómo un componente neuronal puede utilizarse como interfaz entre lenguaje natural y un mecanismo simbólico de planificación.

La arquitectura principal de esta tesis sigue una idea conceptualmente relacionada, aunque no utiliza PDDL ni reproduce directamente la arquitectura LLM+P. En este trabajo, el modelo Qwen2.5 (21), adaptado al dominio de tareas de servicio del robot, recibe una instrucción expresada en lenguaje natural y la transforma en una representación canónica de los objetivos solicitados. Esta representación constituye la interfaz entre el componente neuronal y el componente simbólico de la arquitectura.

El propósito de esta primera etapa es abstraer parte de la variabilidad presente en el lenguaje natural, como diferentes estructuras gramaticales, sinónimos o formas alternativas de expresar una misma intención. De este modo, distintas instrucciones que representan una intención equivalente pueden transformarse en objetivos con una estructura común antes de ser procesadas por el sistema de planificación.

En la segunda etapa, los objetivos obtenidos son enviados a CLIPS. A diferencia del modelo neuronal, CLIPS no interpreta directamente la instrucción original en lenguaje natural. Su función consiste en trabajar sobre los símbolos previamente obtenidos, incorporar los objetivos al estado de conocimiento y aplicar las reglas definidas para descomponerlos y expandirlos en las acciones necesarias para su ejecución. De esta manera, la interpretación lingüística y la expansión simbólica del plan permanecen como responsabilidades diferenciadas dentro de la arquitectura.

Por tanto, aunque LLM+P y la arquitectura propuesta en esta tesis utilizan mecanismos y representaciones diferentes, ambas ilustran una separación entre interpretación neuronal y razonamiento simbólico. En LLM+P, el modelo de lenguaje genera una representación PDDL que posteriormente es procesada por un planificador clásico; en esta tesis, Qwen genera objetivos canónicos que son posteriormente procesados mediante reglas de producción en CLIPS. La semejanza se encuentra, por tanto, en la separación funcional entre ambos paradigmas y no en el formalismo simbólico empleado.

Esta división permite que el modelo neuronal se concentre en el problema de correspondencia entre lenguaje natural y representaciones canónicas, mientras que el conocimiento sobre la estructura de las tareas, sus relaciones y su descomposición se mantiene explícitamente en la base de reglas de CLIPS. En consecuencia, una modificación en las

reglas de planificación no requiere necesariamente modificar el modelo de lenguaje y, de forma análoga, las mejoras realizadas sobre el componente de interpretación pueden introducirse sin sustituir el mecanismo simbólico de planificación.

En esta tesis, por tanto, la integración neuro-simbólica se describe como una composición secuencial: Qwen transforma el lenguaje natural en objetivos canónicos y CLIPS expande dichos objetivos mediante reglas de producción. Los componentes no actúan como dos planificadores concurrentes ni se realiza una fusión automática de planes generados independientemente. El denominado planificador GPT directo se implementa como una arquitectura alternativa con fines de comparación experimental y no como un componente adicional de la cadena neuro-simbólica Qwen-CLIPS.

2.5. Síntesis y conexión con la propuesta

Los antecedentes revisados muestran que los sistemas simbólicos permiten mantener una representación explícita del conocimiento utilizado durante la planificación, mientras que los modelos de lenguaje ofrecen mecanismos para procesar la variabilidad presente en las instrucciones expresadas en lenguaje natural. En una arquitectura híbrida, la combinación de ambos enfoques permite separar el problema de interpretación lingüística del proceso encargado de expandir los objetivos en acciones ejecutables.

Esta tesis estudia dicha separación mediante una comparación controlada entre una arquitectura secuencial Qwen-CLIPS y un planificador GPT directo. Adicionalmente, se utiliza una condición experimental de control en la que los objetivos de referencia son proporcionados directamente a CLIPS. Esta condición permite observar el comportamiento de la etapa simbólica cuando los objetivos de entrada corresponden directamente a la referencia, evitando introducir los errores provenientes de la etapa de interpretación del lenguaje natural. De esta manera, es posible distinguir los errores asociados con la interpretación lingüística de aquellos relacionados con la cobertura y expansión de las reglas de planificación. Esta condición no constituye una tercera arquitectura de planificación, sino un mecanismo auxiliar para aislar los errores correspondientes a la etapa simbólica.

El modelo Qwen2.5-7B ajustado mediante QLoRA recibe una instrucción y produce una secuencia de objetivos canónicos representada en JSON. Un módulo determinista valida y normaliza la respuesta, la convierte en hechos y la envía al motor CLIPS. Finalmente, las reglas de producción expanden cada objetivo en acciones compatibles

con el ejecutor del robot.

Como segunda arquitectura evaluada, se implementa un planificador basado en GPT que genera conjuntamente una representación estructurada de los objetivos interpretados y del plan propuesto. Su salida se restringe mediante JSON Schema y posteriormente se somete a validación determinista. Ambas arquitecturas se evalúan de forma independiente; por tanto, el trabajo no propone una selección concurrente ni una fusión automática de los planes generados.

La contribución experimental consiste en comparar ambos enfoques bajo el mismo conjunto de órdenes, catálogos y acciones, separando los errores de interpretación de los errores de planificación y cuantificando su desempeño mediante métricas de exactitud, ejecutabilidad, consistencia, latencia, uso de recursos y coste.

Capítulo 3

Formalización del problema y arquitectura conceptual

El problema abordado en esta tesis consiste en transformar una instrucción expresada en lenguaje natural en una secuencia de acciones que pueda ser procesada por el sistema de ejecución de un robot de servicio. Esta transformación requiere relacionar dos niveles de representación diferentes: por una parte, la intención expresada por el usuario y, por otra, las acciones concretas disponibles en el robot.

Como se revisó en el capítulo anterior, los modelos de lenguaje proporcionan mecanismos para procesar la variabilidad presente en las instrucciones lingüísticas, mientras que los sistemas simbólicos permiten representar conocimiento y relaciones mediante estructuras explícitas. A partir de estos antecedentes, en este capítulo se formaliza el problema particular considerado en la investigación y se establecen las representaciones utilizadas para describir órdenes, objetivos y planes.

El dominio experimental toma como referencia las tareas de General Purpose Service Robot (GPSR) de RoboCup@Home. Sin embargo, GPSR se utiliza como una fuente controlada de tareas, entidades y escenarios, y no como una restricción que obligue al usuario a reproducir literalmente las formulaciones lingüísticas utilizadas en la competencia.

Finalmente, se presentan las dos arquitecturas que serán evaluadas. La primera utiliza Qwen2.5-7B como intérprete semántico y CLIPS como componente encargado de expandir los objetivos obtenidos en acciones ejecutables. La segunda delega tanto la interpretación como la generación del plan a un modelo GPT. Ambas arquitecturas utilizan el mismo dominio conceptual, pero distribuyen de manera diferente la respon-

sabilidad de transformar una orden en un plan.

3.1. Contexto arquitectónico y alcance del sistema

3.1.1. Integración dentro de la arquitectura ViRBot

La investigación se desarrolla sobre el robot de servicio Justina y su arquitectura de software ViRBot. Esta arquitectura integra diferentes capacidades necesarias para la operación del robot, entre ellas percepción, interacción humano-robot, navegación, manipulación, representación de conocimiento, planificación y ejecución.

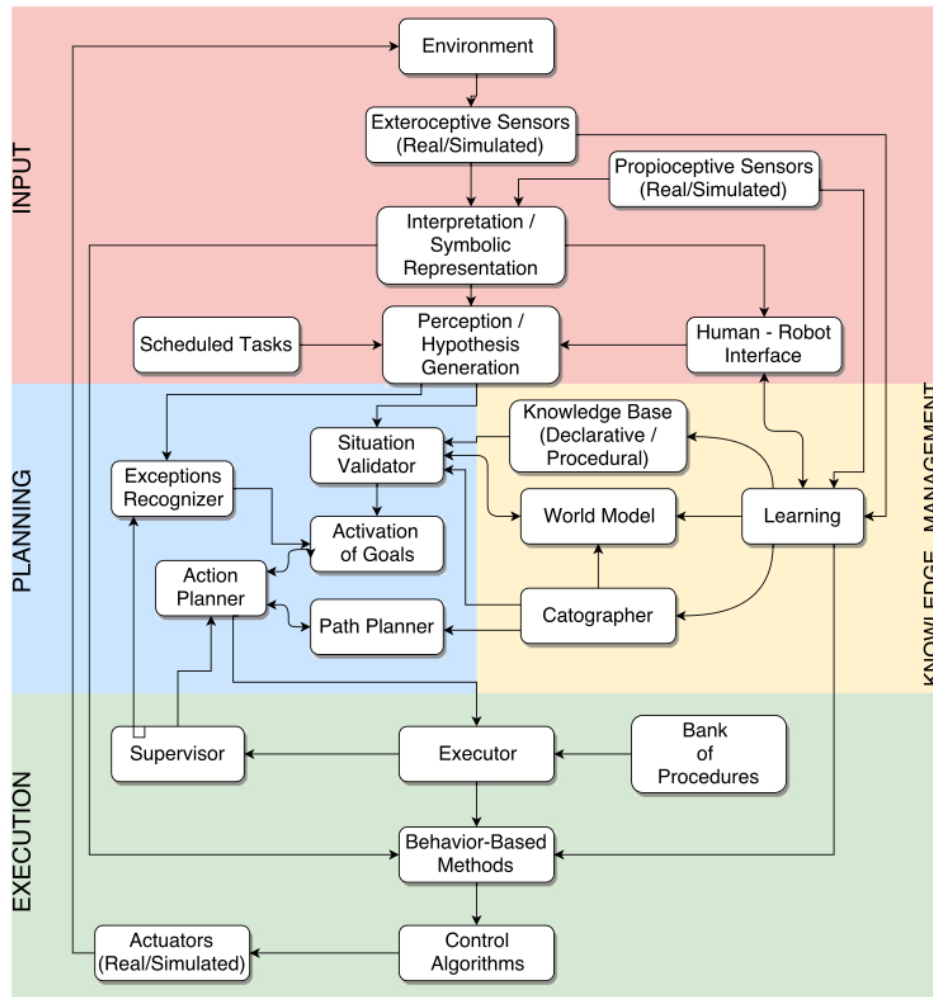


Figura 3.1: Arquitectura general ViRBot del robot Justina.

El sistema desarrollado en esta tesis se ubica entre la recepción de una instrucción

expresada en lenguaje natural y los componentes encargados de ejecutar las acciones correspondientes. Por tanto, no pretende sustituir los módulos de navegación, percepción o manipulación del robot, sino proporcionar una representación ordenada de las acciones que dichos módulos deben realizar.

De manera simplificada, el flujo general puede expresarse como:

$$\boxed{\text{orden en lenguaje natural} \rightarrow \text{interpretación y planificación} \rightarrow \text{plan} \rightarrow \text{ejecución}} \quad (3.1)$$

La contribución estudiada se concentra principalmente en las transformaciones anteriores a la ejecución física. Los módulos existentes de Justina son los responsables de intentar realizar las acciones especificadas en el plan.

3.1.2. Alcance del problema abordado

Una instrucción proporcionada por una persona puede contener diferentes subtareas relacionadas entre sí. Por ejemplo:

“Go to the kitchen, find an apple and bring it to me.”

Esta instrucción contiene información relacionada con navegación, búsqueda, manipulación y entrega. Para que pueda ser procesada por el robot es necesario identificar dichas intenciones y convertirlas posteriormente en operaciones compatibles con sus capacidades.

El problema estudiado puede expresarse inicialmente mediante la transformación:

$$x \rightarrow P, \quad (3.2)$$

donde x representa una orden expresada en lenguaje natural y P el plan generado para atenderla.

Sin embargo, esta representación oculta la diferencia entre comprender la orden y determinar las acciones necesarias para realizarla. Para analizar estas dos funciones por separado, en esta tesis se introduce una representación semántica intermedia G :

$$\boxed{x \rightarrow G \rightarrow P} \quad (3.3)$$

donde:

- x es la orden original;
- G es una secuencia estructurada de objetivos semánticos;
- P es una secuencia de acciones compatibles con el robot.

Esta separación constituye la base conceptual utilizada posteriormente para comparar las arquitecturas Qwen–CLIPS y GPT directo.

3.2. GPSR como dominio de referencia

3.2.1. La prueba General Purpose Service Robot

General Purpose Service Robot (GPSR) es una prueba de RoboCup@Home orientada a evaluar la capacidad de un robot de servicio para recibir instrucciones y resolver tareas que pueden involucrar diferentes capacidades del sistema.

Las instrucciones utilizadas en este tipo de escenario pueden combinar operaciones relacionadas con navegación, búsqueda de personas u objetos, manipulación, entrega, seguimiento, guiado e interacción verbal. Como consecuencia, una sola orden puede requerir que el robot identifique varias subtareas y conserve relaciones entre ellas.

Por ejemplo, una orden puede requerir que el robot se desplace a una habitación, localice un objeto, lo tome y posteriormente lo entregue a una persona. De esta forma, GPSR proporciona un escenario adecuado para estudiar el problema de transformar instrucciones de alto nivel en secuencias de acciones robóticas.

3.2.2. Dominio de referencia y alcance lingüístico

En esta investigación, GPSR se utiliza principalmente como dominio de referencia para definir las familias de tareas y las entidades utilizadas en la generación y evaluación de las instrucciones. A partir de este dominio se establecen catálogos de personas, objetos, categorías, habitaciones y ubicaciones que permiten delimitar el conocimiento utilizado por los sistemas evaluados.

Sin embargo, debe distinguirse entre el dominio de una tarea y la forma lingüística mediante la cual se expresa.

Considérense las siguientes instrucciones:

“Go to the kitchen and find Robin.”

“Could you look for Robin in the kitchen?”

Aunque las expresiones presentan diferencias sintácticas, ambas pueden representar una intención equivalente:

```
go(kitchen)
find(Robin, kind=person)
```

El propósito de la interpretación semántica consiste precisamente en reducir esta variabilidad lingüística y obtener una representación común de la intención.

Por tanto, la entrada del sistema no se restringe a reproducir literalmente las plantillas empleadas para generar las órdenes de referencia. No obstante, la libertad en la formulación lingüística tampoco implica un dominio semántico ilimitado.

Puede establecerse conceptualmente la relación:

$\text{formulación lingüística variable} \rightarrow \text{dominio semántico definido} \rightarrow \text{capacidades disponibles}$
--

(3.4)

En otras palabras, una instrucción puede expresarse de diferentes maneras, pero solamente puede convertirse en un plan cuando su intención puede representarse mediante los objetivos y las capacidades contemplados por el sistema.

El dominio experimental utilizado en esta investigación considera principalmente instrucciones en inglés. Por ello, el estudio de la variabilidad lingüística se concentra en diferentes formas de expresar las tareas soportadas dentro de dicho idioma y no pretende demostrar independencia respecto al lenguaje utilizado.

3.3. Formalización de órdenes, objetivos y planes

3.3.1. Orden en lenguaje natural

Sea:

$$x \in \mathcal{X} \tag{3.5}$$

una instrucción expresada en lenguaje natural perteneciente al conjunto $\mathcal{X}$ de órdenes consideradas dentro del dominio experimental.

La instrucción puede contener una o varias intenciones. Por ejemplo:

“Go to the living room, find Robin and follow Robin to the kitchen.”

contiene tres subtarear diferentes:

1. desplazarse a una ubicación;
2. localizar una persona;
3. realizar una tarea relacionada con dicha persona.

El objetivo de la etapa de interpretación es transformar esta expresión en una representación que elimine información lingüística innecesaria y conserve los elementos relevantes para la planificación.

3.3.2. Entidades y conocimiento del dominio

La interpretación de una orden requiere conocer las entidades que pueden aparecer dentro de las tareas consideradas.

Se define el conocimiento del dominio como:

$$K = \{K_P, K_O, K_C, K_L, K_R\}, \quad (3.6)$$

donde:

- K_P representa el conjunto de personas o nombres conocidos;
- K_O representa el conjunto de objetos;
- K_C representa categorías de objetos;
- K_L representa ubicaciones o lugares relevantes;
- K_R representa habitaciones.

Estos catálogos permiten delimitar las entidades que pueden ser utilizadas en la interpretación de una instrucción y proporcionan información adicional para determinar su función semántica.

Por ejemplo, en:

```
find(Robin, kind=person)
```

Robin corresponde a una entidad de tipo persona, mientras que en:

```
find(apple, kind=object)
```

apple corresponde a un objeto.

Esta información resulta relevante porque una misma estructura de objetivo puede operar sobre tipos de entidades diferentes.

3.3.3. Representación canónica de objetivos

La interpretación de una orden se representa mediante una secuencia ordenada:

$$G = (g_1, g_2, \dots, g_n), \quad (3.7)$$

donde cada g_i representa un objetivo semántico.

De manera general, un objetivo puede expresarse como:

$$g_i = (t_i, \theta_i), \quad (3.8)$$

donde t_i identifica el tipo de objetivo y θ_i contiene sus argumentos y calificadores.

En la representación utilizada por esta investigación, G se serializa mediante una estructura JSON:

```
{
  "goals": [
    "go(living room)",
    "find(Robin, kind=person)",
    "follow(Robin, to=kitchen)"
  ]
}
```

El vocabulario canónico contempla los siguientes tipos principales de objetivos:

```
go
find
take
deliver
place
drop
guide
follow
count
tell
talk
save
```

```
answer_question  
greet
```

Cada objetivo posee una semántica definida y puede requerir parámetros adicionales.
Por ejemplo:

```
go(kitchen)  
find(Robin, kind=person)  
find(apple, kind=object)  
deliver(apple, to=Robin)  
guide(Robin, to=kitchen)
```

Los objetivos `find` y `count` requieren una indicación explícita del tipo de entidad debido a que pueden operar tanto sobre personas como sobre objetos:

```
find(person, kind=person)  
find(apple, kind=object)  
count(person, kind=person)  
count(drinks, kind=object)
```

También pueden utilizarse calificadores para conservar información relevante de la instrucción, por ejemplo:

```
find(person, kind=person, gesture='waving person')  
find(drink, kind=object, property=largest)  
place(apple, on=table)  
follow(Robin, to=kitchen)
```

La representación canónica tiene como objetivo establecer una interfaz estable entre el lenguaje natural y la planificación. Dos expresiones lingüísticas diferentes que describen una misma intención deberían, idealmente, producir la misma secuencia G .

3.3.4. Objetivos semánticos y acciones ejecutables

La representación canónica describe **qué** debe realizar el robot, pero no necesariamente especifica directamente **cómo** debe realizarlo.

Sea:

$$\mathcal{G} \tag{3.9}$$

el vocabulario de objetivos semánticos y:

$$\mathcal{A} \tag{3.10}$$

el repertorio de acciones que pueden ser procesadas por el ejecutor.

En general:

$$\mathcal{G} \neq \mathcal{A}. \tag{3.11}$$

Por ejemplo, `deliver` representa una intención de alto nivel:

```
deliver(apple, to=Robin)
```

pero su realización puede requerir operaciones relacionadas con la localización de la persona, aproximación, entrega del objeto y comunicación.

De manera similar:

```
place(apple, on=table)
```

puede requerir localizar un espacio adecuado antes de liberar el objeto.

Entre las acciones utilizadas por el ejecutor se encuentran, por ejemplo:

```
go_to
find_person
find_pose
approach
focus
find_object
analyze_objects
analyze_person
take
find_space
drop
give
listen
say
```

Por tanto, un objetivo g_i puede producir una subsecuencia de acciones:

$$g_i \rightarrow (a_j, a_{j+1}, \dots, a_k), \quad a_j, \dots, a_k \in \mathcal{A}. \quad (3.12)$$

El plan completo se define entonces como:

$$P = (a_1, a_2, \dots, a_m), \quad a_i \in \mathcal{A}. \quad (3.13)$$

La transformación:

$$G \rightarrow P \quad (3.14)$$

constituye el problema de planificación considerado en esta tesis.

3.3.5. Dependencias y contexto entre objetivos

Los elementos de G no pueden considerarse siempre de manera independiente. Algunas tareas presentan relaciones que deben conservarse durante la interpretación y la planificación.

Por ejemplo:

```
find(apple, kind=object)
take(apple)
deliver(apple, to=Robin)
```

contiene una dependencia entre la adquisición del objeto y su entrega.

Puede representarse mediante una relación de precedencia:

$$take(o) \prec deliver(o, p), \quad (3.15)$$

donde $\prec$ indica que el primer objetivo debe encontrarse antes que el segundo dentro de la secuencia.

De forma similar:

$$take(o) \prec place(o, l). \quad (3.16)$$

La secuencia de objetivos también debe conservar referencias entre entidades. Considérese:

```
find(person, kind=person, gesture='waving person')
guide(person, to=kitchen)
```

El argumento **person** del segundo objetivo debe mantener la referencia a la persona identificada por el objetivo anterior.

Por ello, G no se considera únicamente un conjunto de intenciones, sino una **secuencia ordenada con relaciones de dependencia y contexto**:

$$G = (g_1, \dots, g_n). \quad (3.17)$$

Esta característica resulta especialmente importante en instrucciones compuestas, donde el resultado de una subtarea proporciona información o una condición necesaria para las siguientes.

3.4. Arquitecturas conceptuales evaluadas

A partir de la formalización anterior, las dos arquitecturas estudiadas pueden describirse de acuerdo con la manera en que realizan las transformaciones:

$$x \rightarrow G \quad (3.18)$$

y:

$$G \rightarrow P. \quad (3.19)$$

Las arquitecturas se evalúan de forma independiente y no existe un mecanismo de selección, votación o fusión entre los planes producidos.

3.4.1. Arquitectura Qwen–CLIPS

La primera arquitectura divide explícitamente las funciones de interpretación y planificación.

Qwen2.5-7B recibe una orden x y produce la secuencia canónica de objetivos:

$$I_Q : x \rightarrow G, \quad (3.20)$$

donde I_Q representa el intérprete semántico basado en Qwen.

La secuencia resultante se somete a comprobaciones deterministas antes de ser entregada al componente simbólico. CLIPS recibe posteriormente la representación de dichos objetivos y aplica las reglas correspondientes para producir el plan:

$$R_C : G \rightarrow P, \quad (3.21)$$

donde R_C representa la expansión basada en reglas.

Conceptualmente, la arquitectura puede expresarse como:

$$\boxed{x \xrightarrow{\text{Qwen}} G \xrightarrow{\text{validación y adaptación}} G' \xrightarrow{\text{CLIPS}} P} \quad (3.22)$$

En esta arquitectura, Qwen no genera las acciones primitivas del robot. Su responsabilidad se limita a interpretar la instrucción y producir los objetivos correspondientes. La forma en que cada objetivo se transforma en acciones se encuentra definida en la base de reglas del componente simbólico.

La separación permite distinguir conceptualmente dos posibles fuentes de error:

$$x \rightarrow G \quad \text{e} \quad G \rightarrow P. \quad (3.23)$$

Un plan incorrecto puede originarse porque Qwen interpretó incorrectamente la orden o porque la etapa simbólica no produjo una expansión adecuada para los objetivos recibidos.

La arquitectura se considera local o autoalojada en el sentido de que el modelo Qwen y el componente CLIPS pueden ejecutarse dentro de infraestructura controlada por el sistema, sin requerir un servicio externo de inferencia para cada orden. La configuración física concreta empleada durante los experimentos se describe en el capítulo de implementación.

3.4.2. Arquitectura de planificación directa con GPT

La segunda arquitectura concentra las funciones de interpretación y planificación en un mismo modelo.

Para una orden considerada ejecutable, GPT produce tanto una representación canónica de los objetivos como el plan propuesto:

$$D_{\text{GPT}} : (x, K, \mathcal{G}, \mathcal{A}) \rightarrow (G, P), \quad (3.24)$$

donde K representa el conocimiento del dominio, $\mathcal{G}$ el vocabulario de objetivos permitido y $\mathcal{A}$ el repertorio de acciones disponibles.

Conceptualmente:

$$\boxed{x \xrightarrow{\text{GPT}} (G, P) \xrightarrow{\text{validación}} (G', P')} \quad (3.25)$$

Los objetivos G se conservan como representación auditable de la interpretación realizada por el modelo. Sin embargo, a diferencia de la arquitectura anterior, no son enviados a CLIPS.

Por tanto:

$$G_{\text{GPT}} \not\xrightarrow{\text{CLIPS}} P. \quad (3.26)$$

El mismo modelo debe interpretar la orden y determinar la expansión necesaria para producir acciones pertenecientes a $\mathcal{A}$.

Esta característica permite comparar dos formas diferentes de distribuir la responsabilidad del proceso:

$$\boxed{\begin{array}{l} \text{Qwen-CLIPS: } x \xrightarrow{\text{Qwen}} G \xrightarrow{\text{CLIPS}} P \\ \text{GPT directo: } x \xrightarrow{\text{GPT}} (G, P) \end{array}} \quad (3.27)$$

En ambos casos se conserva una representación semántica de los objetivos, lo que permite evaluar posteriormente la interpretación de forma separada del plan generado.

El contrato completo utilizado por GPT, incluida la posibilidad de solicitar una aclaración o rechazar una instrucción que no pueda convertirse de manera válida en un plan, se desarrolla en los capítulos de metodología e implementación.

3.5. Criterios conceptuales de validez

La generación de una representación estructurada no implica por sí misma que la interpretación o el plan sean correctos. Como se explicó en el capítulo anterior, la conformidad con un formato constituye solamente uno de los niveles de validación.

Para la formalización de esta investigación resulta útil distinguir la validez de la interpretación de la validez del plan.

3.5.1. Validez de la interpretación semántica

Sea:

$$V_I(x, G, K) \quad (3.28)$$

una función que evalúa si la secuencia G representa adecuadamente la instrucción x dentro del conocimiento de dominio K .

Conceptualmente:

$$V_I(x, G, K) \in \{0, 1\}. \quad (3.29)$$

Una interpretación válida debe conservar, entre otros elementos:

- las subtareas solicitadas por la orden;
- el orden lógico de dichas subtareas;
- las entidades involucradas;
- sus tipos semánticos;
- las ubicaciones y destinos;
- los calificadores relevantes;
- las relaciones entre referencias presentes en la instrucción.

Por ejemplo, para:

“Go to the kitchen and find Robin.”

una representación esperada es:

```
go(kitchen)
find(Robin, kind=person)
```

Una salida estructuralmente correcta que omitiera la navegación, cambiara a Robin por otra persona o clasificara a Robin como objeto no representaría correctamente la intención original.

Las métricas concretas utilizadas para determinar esta validez se presentan posteriormente en el capítulo de experimentación.

3.5.2. Validez y consistencia del plan

Sea:

$$V_P(G, P, K, \mathcal{A}) \quad (3.30)$$

una función destinada a evaluar si un plan P constituye una expansión coherente de los objetivos G .

De manera conceptual:

$$V_P(G, P, K, \mathcal{A}) \in \{0, 1\}. \quad (3.31)$$

Un plan válido debe utilizar acciones pertenecientes al repertorio permitido y mantener consistencia con los objetivos que pretende satisfacer.

Entre las propiedades relevantes se encuentran:

- utilización de acciones pertenecientes a $\mathcal{A}$;
- parámetros compatibles con cada acción;
- conservación del orden y las dependencias necesarias;
- inclusión de las subtareas requeridas;
- coherencia entre las entidades utilizadas en objetivos y acciones;
- ausencia de operaciones incompatibles con el estado esperado de la tarea.

Esta separación entre interpretación y planificación permite analizar los errores de manera más precisa.

Puede ocurrir:

$$V_I(x, G, K) = 0, \quad (3.32)$$

aunque la expansión de G sea internamente consistente. En este caso, el problema se encuentra principalmente en la interpretación de la orden.

También puede ocurrir:

$$V_I(x, G, K) = 1 \quad \text{y} \quad V_P(G, P, K, \mathcal{A}) = 0, \quad (3.33)$$

lo que indica que la intención fue identificada correctamente pero el plan generado presenta un problema.

Esta distinción motiva además la utilización posterior de una condición experimental de control en la que los objetivos de referencia son proporcionados directamente a CLIPS:

$$\boxed{G_{\text{ref}} \xrightarrow{\text{CLIPS}} P_{\text{ref}}} \quad (3.34)$$

Esta condición no constituye una tercera arquitectura de planificación. Su función es permitir observar el comportamiento del componente simbólico cuando la entrada semántica se considera correcta, facilitando la separación entre errores de interpretación y errores de planificación.

3.5.3. Planificación y ejecución física

Finalmente, debe distinguirse entre la validez de un plan y el resultado de su ejecución física.

El planificador trabaja con una representación de las acciones que se espera que el robot pueda realizar. Sin embargo, la ejecución se desarrolla en un entorno físico en el que pueden presentarse condiciones que no fueron consideradas durante la generación del plan.

Por tanto:

$$\boxed{\text{plan válido} \not\Rightarrow \text{ejecución física exitosa}} \quad (3.35)$$

Por ejemplo, un plan puede contener correctamente una acción destinada a tomar un objeto, pero la ejecución puede fallar porque el objeto no fue detectado, no se encuentra en la posición esperada o el manipulador no logra sujetarlo.

De manera similar, una acción de navegación puede ser correcta desde el punto de vista del plan y no completarse debido a cambios en el entorno o a una trayectoria bloqueada.

Esta distinción implica que los criterios utilizados para evaluar la interpretación y la planificación no sustituyen la evaluación del desempeño físico del robot. La ejecución sobre Justina constituye un nivel adicional de validación que se describe posteriormente en el diseño experimental.

En consecuencia, el problema central de esta tesis queda definido mediante tres representaciones:

$$\boxed{x \rightarrow G \rightarrow P \rightarrow \text{ejecución}} \quad (3.36)$$

Las dos arquitecturas evaluadas difieren principalmente en el mecanismo utilizado

para realizar las dos primeras transformaciones. Qwen-CLIPS separa la interpretación lingüística de la expansión simbólica del plan, mientras que GPT directo concentra ambas funciones en un único modelo generativo. Esta diferencia proporciona la base conceptual para el diseño metodológico y la comparación experimental desarrollados en los capítulos siguientes.

Capítulo 4

Metodología: Diseño del Sistema Híbrido CLIPS-ChatGPT/Qwen

4.1. Diseño del Sistema Basado en Reglas con CLIPS

4.1.1. Definición del Conjunto de Reglas Jerárquicas

Especificación de las reglas organizadas por niveles de prioridad, donde las reglas de seguridad (evitación de colisiones, gestión de emergencias) tienen máxima prioridad sobre las operativas.

4.1.2. Priorización de Acciones Críticas y Gestión de Emergencias

Diseño de reglas específicas para escenarios de fallo y situaciones críticas, asegurando respuestas predecibles y seguras.

4.1.3. Representación del Conocimiento y Hechos en la Base de Datos de CLIPS

Estructuración del conocimiento del entorno doméstico en hechos CLIPS iniciales que sirven como base para el razonamiento.

Descripción de la base de conocimiento inicial incluye:

- *Topología del entorno*: habitaciones, conexiones, zonas navegables.
- *Taxonomía de objetos*: categorías, subcategorías, propiedades heredables.
- *Relaciones espaciales*: contención, adyacencia, orientación.
- *Capacidades del robot*: acciones posibles, restricciones físicas.

4.2. Integración de ChatGPT/Qwen como Sistema Complementario

4.2.1. Funciones Asignadas al Módulo de Lenguaje Natural

4.2.2. Protocolo de Comunicación entre CLIPS y los Modelos de Lenguaje

4.2.3. Mecanismos de Seguridad y Validación para las Respuestas del LLM

Implementación de verificaciones para asegurar que los planes generados por LLMs cumplen con constraints de seguridad y factibilidad robótica.

4.3. Implementación de Planificación Dual

4.3.1. Diseño del Módulo de Planificación con ChatGPT

4.3.2. Protocolos de Prompting para Generación de Planes Robóticos

4.3.3. Mecanismo de Selección entre Planes CLIPS vs. ChatGPT

4.3.4. Validación y Simulación de Planes Antes de Ejecución

Integración con el Robot Justina

- *Procesamiento de Entrada de Voz*.
- *Ejecución de Planes en el Entorno Físico*.

- Mecanismos para transformar los planes publicados en los tópicos de resultados en comandos ejecutables por los actuadores del robot (navegación, manipulación de objetos, etc.)
- *Retroalimentación y Aprendizaje Incremental.*

Capítulo 5

Implementación

- 5.1. Entornos de Desarrollo: Simulación y Plataforma Física
- 5.2. Implementación del Motor de Reglas en CLIPS
- 5.3. Desarrollo del Módulo de Integración
- 5.4. Configuración de la Interfaz con API de OpenAI para ChatGPT o Modelo Qwen Local
- 5.5. Casos de Prueba para Validar la Interacción entre los Módulos

Capítulo 6

Escenarios de Validación y Experimentación

6.1. Diseño de Experimentos en Entornos Controlados

6.1.1. Escenario 1: Ejecución de Tareas Predefinidas (solo CLIPS)

6.1.2. Escenario 2: Gestión de Órdenes Imprecisas o Novedosas (CLIPS + ChatGPT/Qwen)

6.1.3. Escenario 3: Respuesta a Eventos Inesperados o Fallos

6.1.4. Escenario 4: Planificación Dual para Tareas Complejas (CLIPS vs. ChatGPT)

- Comparativa de eficiencia en generación de planes
- Evaluación de robustez ante escenarios novedosos
- Análisis de seguridad en planes generados por LLMs

6.2. Métricas de Evaluación

- 6.2.1. Métricas Cuantitativas: Tiempo de Ejecución, Tasa de Éxito, Uso de Recursos Computacionales**
- 6.2.2. Métricas Cualitativas: Robustez, Interpretabilidad y Fluidez en la Interacción Humano-Robot**

Capítulo 7

Análisis de Resultados

- 7.1. Comparativa del Rendimiento del Sistema Solo-CLIPS vs. el Sistema Híbrido
- 7.2. Evaluación de la Efectividad de ChatGPT/Qwen en las Diferentes Funciones Asignadas
- 7.3. Discusión de Limitaciones y Errores
- 7.4. Análisis de la Escalabilidad y el Coste Computacional de la Integración

Capítulo 8

Discusión

- 8.1. Interpretación de los Resultados en el Contexto de los Objetivos
- 8.2. Ventajas y Desventajas del Enfoque Híbrido Propuesto
- 8.3. Implicaciones para la Seguridad y Certificación en Entornos Regulados
- 8.4. Límites Éticos y Prácticos del Uso de ChatGPT/Qwen en Robótica

Capítulo 9

Contribuciones y Relevancia

- 9.1. Contribución Técnica: Marco Híbrido Escalable y Documentado
- 9.2. Relevancia Práctica: Aplicaciones en Salud, Industria y Logística
- 9.3. Relevancia Académica: Benchmark para Sistemas Rule-Based vs. Híbridos

Capítulo 10

Conclusiones y Trabajo Futuro

10.1. Conclusiones Principales

10.2. Propuestas de Trabajo Futuro

- Fine-tuning de un LLM de código abierto para dominio específico
- Mejora de los mecanismos de seguridad y verificación
- Exploración de arquitecturas de integración más profundas

Referencias

- [1] R. R. Murphy, *Introduction to AI Robotics*. MIT Press, 2nd ed., 2019.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson, 4th ed., 2020.
- [3] J. C. Giarratano and G. D. Riley, *Expert Systems: Principles and Programming*. Thomson Course Technology, 4th ed., 2005.
- [4] R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE Journal of Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [6] L. Shi, X. Chen, Y. Hu, J. Li, H. Zhang, and H. Zhao, “Chatgpt for robotics: Design principles and model abilities,” *Microsoft Research*, 2023.
- [7] NASA Johnson Space Center, “Clips reference manual,” tech. rep., NASA, 1991.
- [8] B. Pell *et al.*, “An autonomous spacecraft agent prototype,” in *First International Conference on Autonomous Agents*, pp. 253–261, 1997.
- [9] J. Savage and otros, “Virbot: A modular architecture for service robots with hybrid planning,” in *RoboCup Symposium*, 2024.
- [10] L. Cheng, Y. Mao, and T. Ward, “Editorial: Neural network models in autonomous robotics,” *Frontiers in Neurorobotics*, 2025.
- [11] R. Raj and A. Kos, “An extensive study of convolutional neural networks: Applications in computer vision for improved robotics perceptions,” *Sensors*, 2025.
- [12] A. Sadek, *Building autonomous agents with hybrid navigation policies*. PhD thesis, Université de Technologie de Compiègne, 2024.

- [13] Alibaba Cloud Qwen Team, “Qwen official github repository,” 2026.
- [14] OpenAI, “Openai developer documentation and model guides,” 2026.
- [15] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for NLP,” in *Proceedings of the 36th International Conference on Machine Learning*, vol. 97 of *Proceedings of Machine Learning Research*, pp. 2790–2799, PMLR, 2019.
- [16] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [17] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems*, vol. 36, pp. 10088–10115, 2023.
- [18] A. d’Avila Garcez, T. R. Besold, L. De Raedt, P. Földiák, P. Hitzler, T. Icard, K.-U. Kühnberger, L. C. Lamb, R. Miikkulainen, and D. L. Silver, “Neural-symbolic learning and reasoning: Contributions and challenges,” in *Knowledge Representation and Reasoning: Integrating Symbolic and Neural Approaches: Papers from the 2015 AAAI Spring Symposium*, pp. 18–21, AAAI Press, 2015.
- [19] T. R. Besold, A. d’Avila Garcez, S. Bader, H. Bowman, P. Domingos, P. Hitzler, K.-U. Kuehnberger, L. C. Lamb, D. Lowd, P. M. V. Lima, L. de Penning, G. Pinkas, H. Poon, and G. Zaverucha, “Neural-symbolic learning and reasoning: A survey and interpretation,” *arXiv preprint arXiv:1711.03902*, 2017.
- [20] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, “LLM+P: Empowering large language models with optimal planning proficiency,” *arXiv preprint arXiv:2304.11477*, 2023.
- [21] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, *et al.*, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [22] D. Nau, T.-C. Au, O. Ilghami, U. Kuter, J. W. Murdock, D. Wu, and F. Yaman, “Shop2: An htn planning system,” *Journal of Artificial Intelligence Research*, vol. 20, pp. 379–404, 2003.

- [23] K. Erol, J. Hendler, and D. S. Nau, “Htn planning: Complexity and expressivity,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 1123–1128, 1994.
- [24] I. Georgievski and M. Aiello, “An overview of hierarchical task network planning,” *Journal of Autonomous Agents and Multi-Agent Systems*, vol. 30, no. 4, pp. 709–752, 2016.
- [25] R. E. Fikes and N. J. Nilsson, “Strips: A new approach to the application of theorem proving to problem solving,” *Artificial Intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.
- [26] H. Geffner, “Model-free, model-based, and general intelligence,” *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 10–17, 2020.
- [27] R. Lallement, L. de Silva, and R. Alami, “Hatp: An htn planner for robotics,” in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, pp. 10–18, 2014.
- [28] L. de Silva, R. Lallement, and R. Alami, “The hatp hierarchical planner: Formalisation and an initial study of its usability and practicality,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5845–5852, 2016.
- [29] L. de Silva and R. Alami, “Dealing with on-line human-robot negotiations in hierarchical agent-based task planner,” in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, pp. 67–75, 2017.
- [30] R. Alami, R. Chatila, A. Clodic, *et al.*, “On human-aware task and motion planning,” *Robotics and Autonomous Systems*, vol. 61, no. 8, pp. 804–813, 2013.
- [31] X. Luo, S. Xu, R. Liu, and C. Liu, “Decomposition-based hierarchical task allocation and planning for multi-robots under hierarchical temporal logic specifications,” *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6142–6149, 2024.
- [32] E. Gat, “On three-layer architectures,” in *Artificial Intelligence and Mobile Robots*, pp. 195–210, MIT Press, 1998.
- [33] F. Morales *et al.*, “Automated planning and scheduling in robot-aided rehabilitation: A review,” *Journal of NeuroEngineering and Rehabilitation*, vol. 22, no. 1, 2025.

- p. 45, 2025.
- [34] W. Pang, H. Li, Q. Huang, P. Li, and X. Ma, “Ontology-based mission planning for unmanned systems: A review,” *Journal of Unmanned Systems*, vol. 10, no. 2, pp. 112–128, 2022.
 - [35] Middlesex University, “Recognise act cycle,” 2023.
 - [36] B. L. Donnell, “Object/rule integration in clips,” *Expert Systems*, vol. 11, no. 1, pp. 29–45, 1994.
 - [37] F. J. Fanning, “An evaluation of an ada implementation of the rete algorithm for embedded flight processors,” *Air Force Institute of Technology Theses and Dissertations*, 1990.
 - [38] University of Žilina, “Production rules,” 2023.
 - [39] ScienceDirect, “Production systems and rules,” 2023.
 - [40] C. L. Forgy, “Rete: A fast algorithm for the many pattern/many object pattern match problem,” *Artificial Intelligence*, vol. 19, no. 1, pp. 17–37, 1982.
 - [41] C. L. Forgy, *On the efficient implementation of production systems*. PhD thesis, Carnegie-Mellon University, 1979.
 - [42] A. Zeng, P. Florence, J. Tompson, S. Welker, J. Chien, M. Attarian, T. Armstrong, I. Krasin, D. Duong, V. Sindhwani, and J. Lee, “Robotic pick-and-place with natural language commands,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 4615–4622, 2020.
 - [43] L. Albert, “Average case complexity analysis of rete pattern-match algorithm, and average size of join in databases,” in *Foundations of Software Technology and Theoretical Computer Science*, vol. 405 of *Lecture Notes in Computer Science*, pp. 223–241, Springer, 1989.
 - [44] GitHub, “Attention is all you need - paper reading notes,” 2023.
 - [45] C. de desarrolladores de Alibaba Cloud, “Transformer modelos de lenguaje de nuestro tiempo: El desarrollo y la evolución tecnológica de los modelos de lenguaje a gran escala,” 2026.
 - [46] systems analysis.ru, “Transformer-architektur - der transformer,” 2025.
 - [47] A. Kashyap, “Recurrent memory-augmented transformers with chunked attention for long-context language modeling,” *arXiv preprint arXiv:2507.00453*, 2025.

- [48] D. Pollini, B. V. Guterres, R. S. Guerra, and R. B. Grando, “Reducing latency in llm-based natural language commands processing for robot navigation,” 2025.
- [49] C. Blog, “Qwen2.5-0.5b caso práctico: Guía para la construcción de un robot guía inteligente.,” 2026.